DATA STRUCTURE AND ALGORITHMS

LESSON 04 – LINKED LIST

Conceptual Difficulty

Implementation Difficulty

LECTURER: SEAK LENG

OVERVIEW

1. Introduction to algorithm
2. Basic data types and statements
3. Control structures and Loop
4. Array
5. Data structure
6. Sub-programs
7. Recursive
8. Pointers
9. *Linked Lists*
10. Stacks and Queues
11. Sorting algorithms
12. Trees

C++

OUTLINE

- What is linked list?
  - Single linked list? Double linked list?
- Linked list Vs. Array
- How to implement linked list in C++
  - Insert, Search, Update
- Examples

ARRAY VS. LINKED LIST

Array

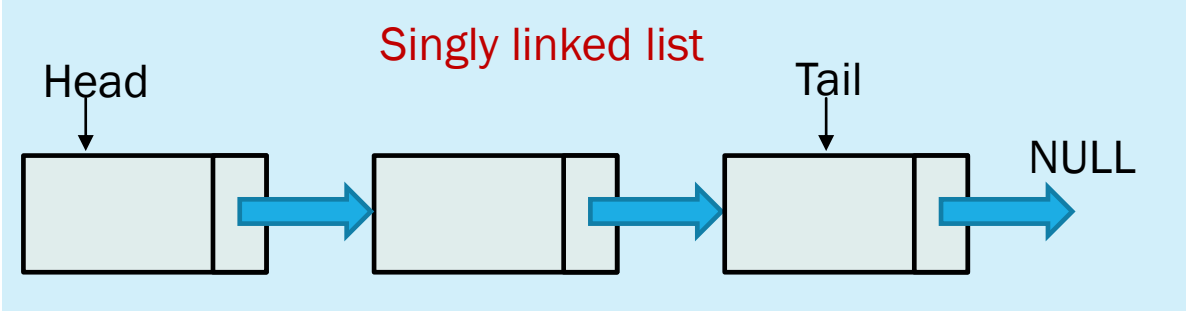
- Limited amount of elements
- Fixed size
- Once created, can't add or reduce number of elements to be stored
- Can random access
- Faster access (Elements in contiguous memory locations)

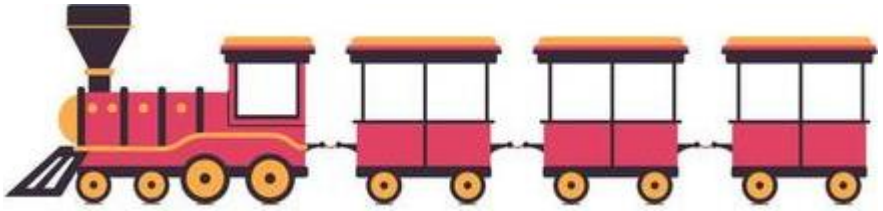
Linked List

- Unlimited amount of elements
- Dynamically shrink and grow
- Dynamic memory management
- No random access is allowed
- Slower access
  - Elements not in contiguous memory locations

WHAT IS LINKED LIST?

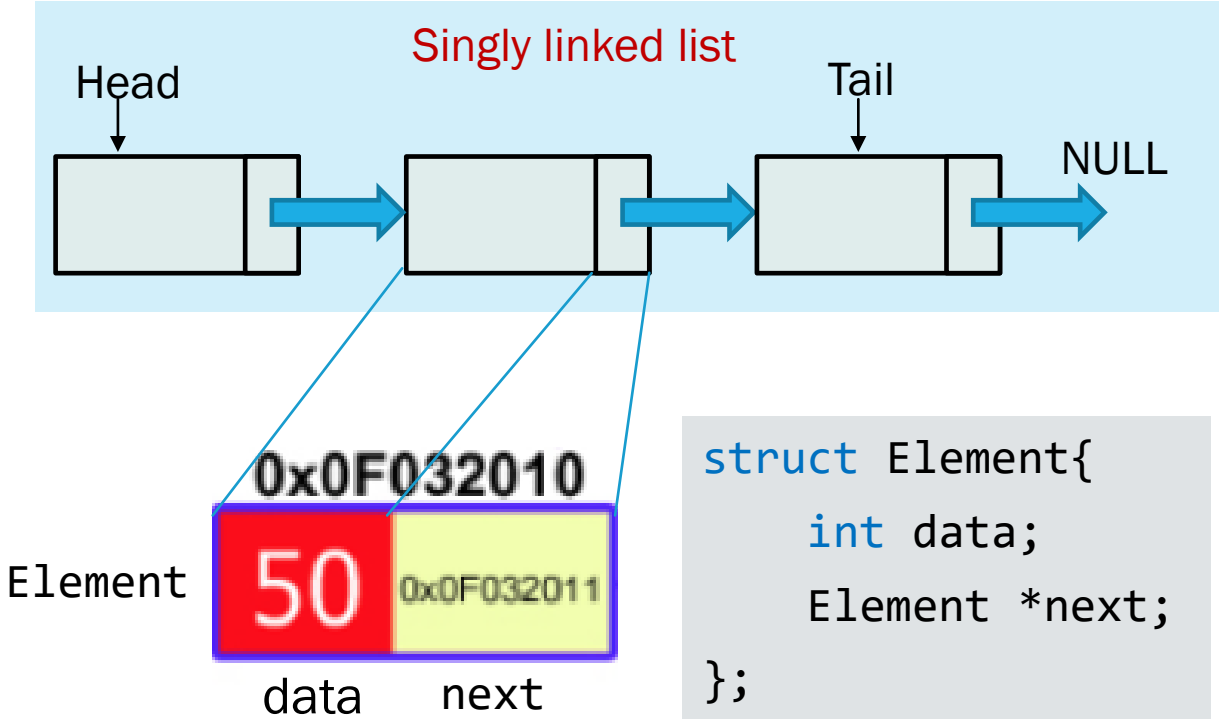
- A linked list is a linear data structure where elements are stored in nodes, and each node points to the next node in the sequence. Unlike arrays, linked lists don't require contiguous memory allocation.
- **Linked list** can store an indefinite number of elements/nodes (dynamic size).
- In a linked list, each element is linked with each other.





WHAT IS LINKED LIST?

- In a linked list, each element/node is linked with each other. Accessing to a linked list is done sequentially.
- Each element contains at least:
  - ✓ Data (can be any data type)
  - ✓ Ponter (link): a reference to its next element (**successor**) and/or previous element (**predecessor**)




```
struct Element{
    int data;
    Element *next;
};
```

C++ Procedural Code for Singly Linked List

WHAT IS LINKED LIST?

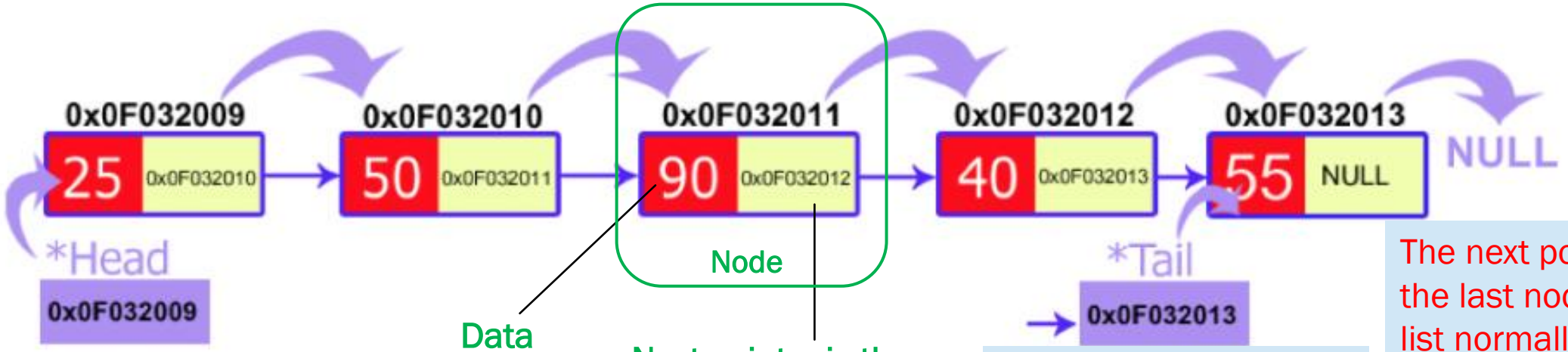
- *Node:* Contains data and a pointer to the next node
- *Head:* Pointer to the first node in the list, compulsory.
- *Tail:* Pointer to the last node in the list (optional, but useful)
- *N:* Number of nodes in the list (optional, but useful)

C++ Procedural Code for Singly Linked List:

```
struct Element{
    int data;
    Element *next;
};
```

```
struct List{
    int n;
    Element *head;
    Element *tail;
};
```

Example of Singly Linked List



Head is the pointer of the first node of the list

Next pointer is the Address of next element

Tail is the pointer of the last node of the list

The next pointer of the last node of the list normally points to NULL

WHY USE A LINKED LIST?

Advantages:

- Easy to **add or remove** items (no need to shift everything like in arrays).
- **Flexible size** – it can grow or shrink easily.

Disadvantages:

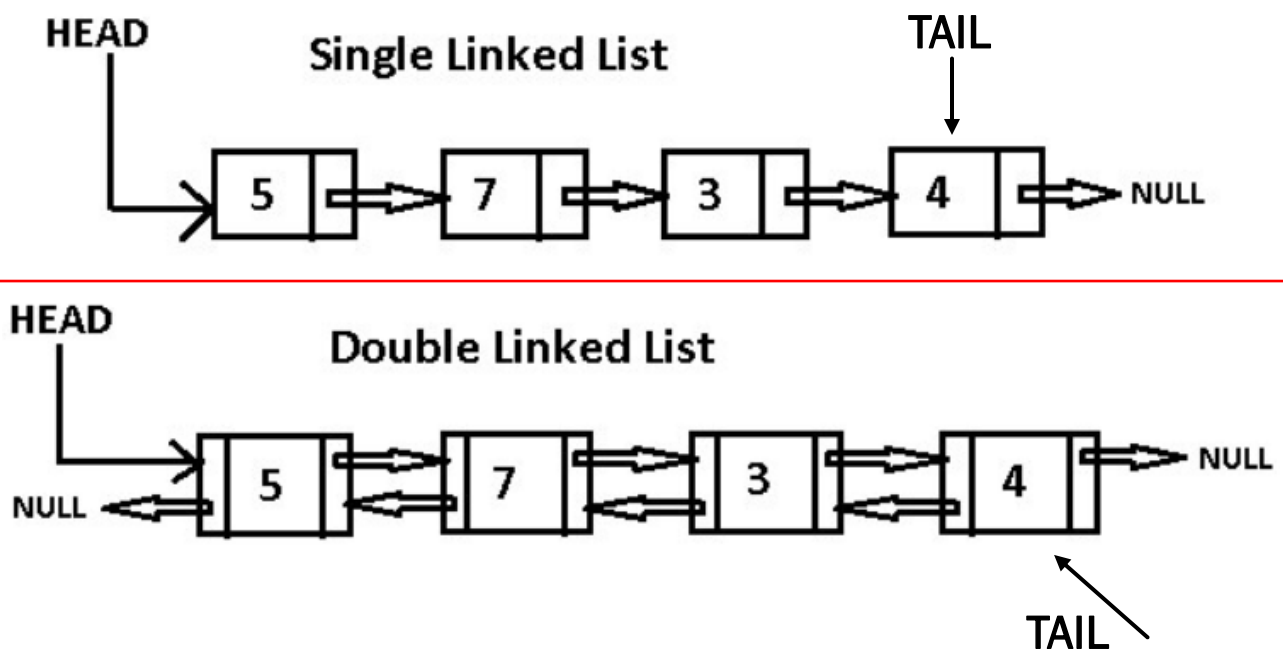
- **Slower access** – you have to go node by node to find something (no random access like arrays).

TYPES OF LINKED LIST

Successor : Next element
Predecessor : Previous element

- There are two main types of linked lists:
  1. A singly linked list is a linked list that has **a link to either its successor or predecessor**. Each node points only to the next node.
  2. A doubly linked list is a linked list that has **both links** to successor and predecessor.

```
struct List{
    int n;
    Element *head;
    Element *tail;
};
```




```
struct Element{
    int data;
    Element *next;
};
```

```
struct Element{
    int data;
    Element *next;
    Element *previous;
};
```

LIST OPERATIONS

- ✓Creating a list
- ✓Insert a new element to a list
- ✓Search an element in a list
- ✓Update an element in a list
- ✓Delete an element from a list
- ✓Reverse a list
- ✓Sort a list using pointer manipulation
- ✓Combine two lists

Singly Linked List (SLL)

SINGLY LINKED LIST

- Create a linked list

1. Define a node structure: data, next pointer
2. Define a list structure: head, tail, n
3. Create an empty list

Steps to create an empty list:

1. Create a list variable
2. Allocate memory
3. Set 0 to n since we are creating an empty list
4. Head points to **NULL or nullptr**
5. Tail points to **NULL or nullptr**

```cpp
#include<iostream>
using namespace std;
struct Element{
    int data;
    Element *next;
};
```

```cpp
struct List{
    int n;  //number of elements
    Element *head;
    Element *tail;
};
```

```cpp
//A function to create an empty list
List* createList(){
    List *ls;

    ls = new List(); //allocate memory
    //ls.n = 0; //error
    ls->n = 0;
    ls->head = NULL;
    ls->tail = NULL;

    return ls;
}
```

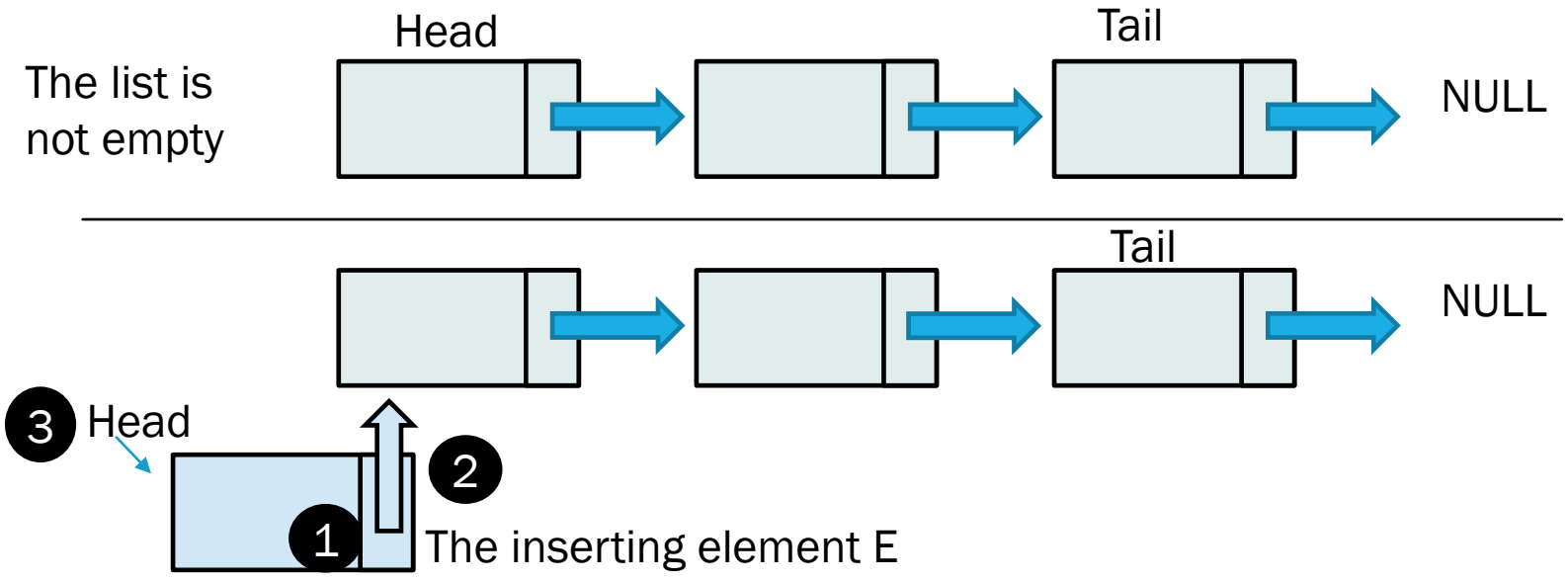
C++ Procedural Code for Singly Linked List

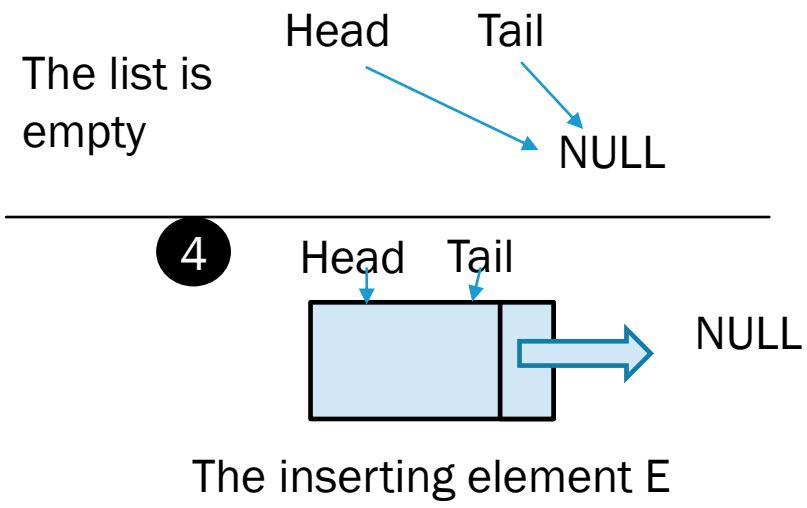
SINGLY LINKED LIST

- Insert a new element to a list
  - Insert at the beginning
  - Insert at the end
  - Insert at any position

INSERTION AT THE BEGINNING OF THE LIST

1. Create an element E to be inserted at the beginning of the linked list
   - If the list is not empty
     2. Make the next pointer of E points to the first node (Head)
     3. Make the head pointer points to E
4. If the list is empty, make Head and Tail points to E and next pointer of Head and Tail points to NULL





INSERTION AT THE BEGINNING OF THE LIST

Steps to insert element at the beginning of a linked list

1. Create a new element E
2. Make next pointer of E points to head of list
3. Update E to be head of list
4. Update tail if the list is empty
5. Increase n by 1

Pseudocode: Function to Add Element at the beginning of SLL

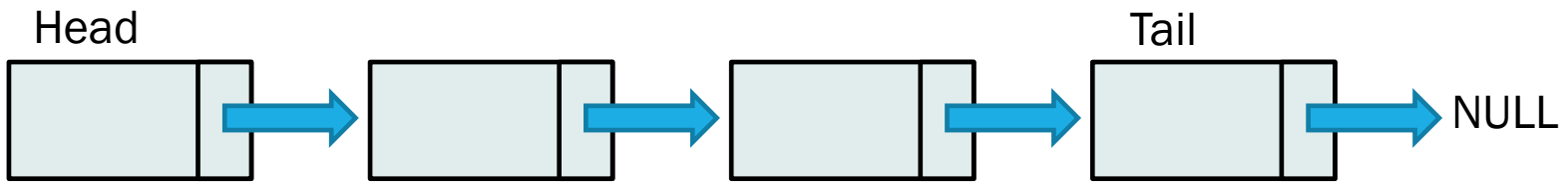
```
Procedure insert_be(*ls: List, d: data_type)
        var  *E: Element
        E ← new(size(Element))
        E→data ← d

        E→next ← ls→head
        ls→head ← E
        if(ls→n ==0) then
            ls→tail ← E
        end if
        ls→n ← ls→n + 1
End procedure
```

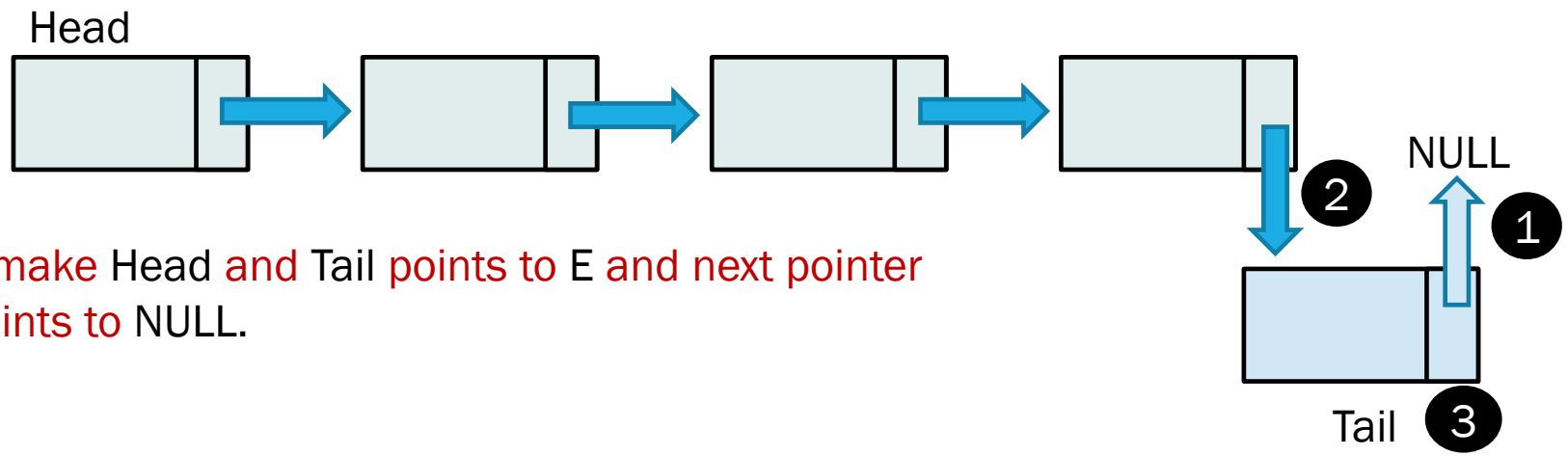
```
void add_beg(List* ls, int data){
    Element* e = new Element;
    e->data = data;
    e->next = ls->head;
    ls->head = e;

    if(ls->n == 0){
        ls->tail = e;
    }

    ls->n++;
}
```

INSERTION AT THE END OF THE LIST

1. Create an element E to be inserted at the end of the linked list, next pointer of E to **NULL**
2. If the list is not empty, next pointer of tail points to E.
3. Simply make the last element (tail) points to E

If the list is not empty





4. If the list is empty, make Head and Tail points to E and next pointer of Head and Tail points to NULL.

INSERTION AT THE END OF THE LIST

Pseudocode: Function to Add Element to End of SLL

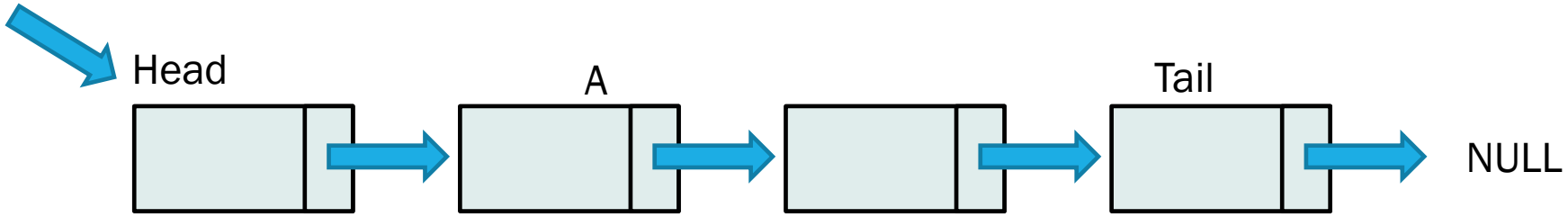
```
Procedure insert_end(*ls: List, d: data_type)
        var *E: Element
        E ← new(size(Element))
        E→data ← d
        E→next ← null

        if (ls→n == 0) then
                ls->head = E
        else
                ls→tail→next ← E
        end if
                ls→tail ← E
                ls→n ← ls→n + 1
End procedure
```

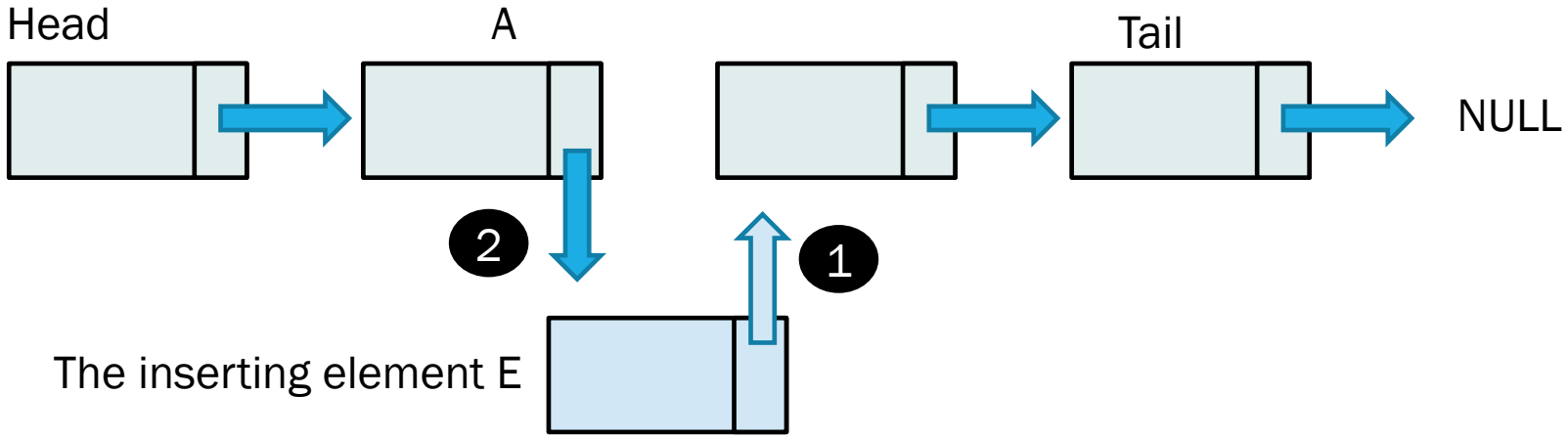
```
void add_end(List* ls, int data){
    Element* e = new Element;
    e->data = data;
    e->next = nullptr;

    if(ls->n == 0){
        ls->head = e;
    }else{
        ls->tail->next = e;

    }
    ls->tail = e;
    ls->n ++;
}
```

INSERTION AFTER THE ELEMENT A

1. Make the pointer of E points to the element after A
2. Make the pointer of A points to E

If the list is not empty





INSERTION AFTER THE ELEMENT AT POSITION

- If pos is out of bounds (negative or >= n), show error.
- If the position is valid, traverse the list until reaching position (i<pos):
  - Set newNode.next to current.next
  - Set current.next to newNode
  - Update tail if inserted at the end (current == ls->tail)

```
void add_after_pos(List* ls, int data, int pos){
    if(pos<0 || pos >= ls->n){
        cout << "Cannot add at this position\n";
        return;
    }

    Element* current = ls->head;
    for(int i=0;i<pos;i++){
        current = current->next;
    }

    Element* e = new Element;
    e->data = data;
    e->next = current->next;
    current->next = e;

    if(current == ls->tail){
        ls->tail = e;
    }

    ls->n++;
}
```

INSERTION BEFORE THE ELEMENT AT POSITION

- If pos is out of bounds (negative or > n), show error.
- If pos == 0, it's the same as add_beg().
- If pos > 0 && pos <= n , traverse to the node before pos (i<pos-1), then insert.
  - Set newNode.next to current.next
  - Set current.next to newNode
  - Update tail if inserted at the end (newNode->next == NULL)

```
void add_before_pos(List* ls, int data, int pos){
    if(pos < 0 || pos > ls->n){
        cout << "Cannot add before this position\n";
        return;
    }
    Element* e = new Element;
    e->data = data;

    if(pos == 0){ // Add before head
        e->next = ls->head;
        ls->head = e;
        if(ls->n==0){
            ls->tail = e;
        }
    }else{
        Element* current = ls->head;
        for(int i=0;i<pos-1;i++){
            current = current->next;
        }
        e->next = current->next;
        current->next = e;
        if(e->next == nullptr){
            ls->tail = e;
        }
    }
    ls->n++;
}
```

DISPLAY ELEMENTS IN THE SINGLY LINKED LIST

Steps to display element in list

1. Start from the head node:
   - Assign the head node to a temporary pointer
2. Traverse the list: Use a loop to go through each node until the pointer becomes NULL.
3. Print each node's data:
   - Inside the loop, print the data of the current node.
4. Move to the next node:
   - Update the pointer to the next node (current = current->next).

Pseudocode for function to display elements in the singly linked list

```
Procedure void(*ls: List)
        var *tmp: Element
        tmp ← ls→head
        while(tmp!=NULL) do
                write(tmp→data)
                tmp ← tmp→next
        end while
End procedure
```

```cpp
void displayList(List *ls){
    Element *tmp; //temporary variable

    tmp = ls->head;
    while(tmp!=NULL){
        cout<<tmp->data<<" ";
        tmp = tmp->next;
    }
    cout<<endl;
}
```

C++ Procedural Code

IMPLEMENTATION OF SINGLY LINKED LIST IN C++

```cpp
#include<iostream>
using namespace std;
struct Element{
    int data;
    Element *next;
};
```

```cpp
struct List{
    int n;  //number of elements
    Element *head;
    Element *tail;
};
```

```cpp
//A function to create an empty list
List* createList(){
    List *ls;

    ls = new List(); //allocate memory
    //ls.n = 0; //error
    ls->n = 0;
    ls->head = NULL;
    ls->tail = NULL;

    return ls;
}
```

```cpp
void insert_begin(List *ls, int newData){
    //Create new element
    Element *e;
    e = new Element();
    e->data = newData;

    //Update pointer, head, tail
    e->next = ls->head;
    ls->head = e;
    if(ls->n == 0){
        ls->tail = e;
    }
    ls->n = ls->n + 1;
}
```

```cpp
void displayList(List *ls){
    Element *tmp; //temporary variable

    tmp = ls->head;
    while(tmp!=NULL){
        cout<<tmp->data<<" ";
        tmp = tmp->next;
    }
    cout<<endl;
}
```

```cpp
int main(){

    List *L;
    L = createList();

    insert_begin(L, 3);
    insert_begin(L, 2);
    insert_begin(L, 5);
    displayList(L);
    displayList(L);
    displayList(L);
    cout<<L->n<<endl;
}
```

Output

```
5 2 3
5 2 3
5 2 3
3
```

C++ Procedural Code for Inserting elements at the beginning of the Singly Linked List

C++ Procedural Code for Inserting elements in the Singly Linked List

```cpp
#include <iostream>
using namespace std;

struct Element {
    int data;
    Element* next;
};

struct List{
    int n;
    Element* head;
    Element* tail;
};

List* createEmptyList(){
    List* ls = new List;
    ls->head = nullptr;
    ls->tail = nullptr;
    ls->n = 0;
    return ls;
}
```

```cpp
void add_beg(List* ls, int data){
    Element* e = new Element;
    e->data = data;
    e->next = ls->head;
    ls->head = e;

    if(ls->n == 0){
        ls->tail = e;
    }

    ls->n++;
}
```

```cpp
void add_end(List* ls, int data){
    Element* e = new Element;
    e->data = data;
    e->next = nullptr;

    if(ls->n == 0){
        ls->head = e;
    }else{
        ls->tail->next = e;
    }
    ls->tail = e;
    ls->n++;
}
```

```cpp
void add_after_pos(List* ls, int data, int pos){
    if(pos<0 || pos >= ls->n){
        cout << "Cannot add at this position\n";
        return;
    }

    Element* current = ls->head;
    for(int i=0;i<pos;i++){
        current = current->next;
    }

    Element* e = new Element;
    e->data = data;
    e->next = current->next;
    current->next = e;

    if(current == ls->tail){
        ls->tail = e;
    }

    ls->n++;
}
```

```cpp
void add_before_pos(List* ls, int data, int pos){
    if(pos < 0 || pos > ls->n){
        cout << "Cannot add before this position\n";
        return;
    }
    Element* e = new Element;
    e->data = data;

    if(pos == 0){ // Add before head
        e->next = ls->head;
        ls->head = e;
        if(ls->n==0){
            ls->tail = e;
        }
    }else{
        Element* current = ls->head;
        for(int i=0;i<pos-1;i++){
            current = current->next;
        }
        e->next = current->next;
        current->next = e;
        if(e->next == nullptr){
            ls->tail = e;
        }
    }
    ls->n++;
}
```

```cpp
void display(List* ls){
    Element * t = ls->head;
    while(t != nullptr){
        cout << t->data << " ";
        t = t->next;
    }
    cout << endl;
}
```

```cpp
int main(){
    List* list1 = createEmptyList();
    for(int i=0;i<10;i++){
        add_beg(list1, i);
    }
    display(list1);

    List* list2 = createEmptyList();
    for(int i=0;i<10;i++){
        add_end(list2, i);
    }
    display(list2);

    List* list3 = createEmptyList();
    add_before_pos(list3,777,0);
    display(list3);
    add_before_pos(list1,999,2);
    display(list1);

}
```

SEARCH DATA IN SINGLY LINKED LIST

To Find the position of a node with a given value.

Steps:

1.Initialize a pointer current to the head of the list.

2.Initialize a variable position to 0.

3.Repeat until current is NULL:

- •If current->data == target, return position.
- •Move current to current->next.
- •Increment position by 1.

4.If loop finishes without finding the value, return -1 (not found).

```
int searchData(List* ls, int data){
    Element* e = ls->head;
    int position = 0;
    while(e != nullptr){
        if(e->data == data){
            cout << data << " is found at " ;
            return position;
        }
        position++;
        e = e->next;
    }
    cout << data << " is not found " ;
    return -1;
}
```

UPDATE DATA IN THE LINKED LIST

Update the first occurrence of a value in the list with a new value.

Steps:

1.Initialize a pointer current to the head of the list.

2.Repeat until current is NULL:

- If current->data == oldValue, set current->data = newValue and exit.
- Move current to current->next.

3.If loop finishes without finding the old value, display a message like "Value not found".

```
void updateData(List* ls, int oldData, int newData){
    Element* e = ls->head;
    int isFound = 0;
    while(e!=nullptr){
        if(e->data == oldData){
            e->data = newData;
            isFound = 1;
        }
        e = e->next;
    }
    if(isFound){
        cout << "Data is updated\n";
    }else{
        cout << "No data is updated\n";
    }
}
```

PRACTICE

- What is the output of the program?

```
#include <iostream>
using namespace std;

struct Person{
    int id;
    string name;
};

struct Element{
    Person data;
    Element* next;
};

struct List{
    int n;
    Element* head;
    Element* tail;
};
```

```
void function1(List* ls, Person data){
    Element* e = new Element;
    e->data = data;
    e->next = ls->head;
    ls->head = e;
    if(ls->n==0){
        ls->tail = e;
    }
    ls->n++;
}
```

```
void display(List* ls){
    Element* e = ls->head;
    while(e != nullptr){
        cout << e->data.id << " " << e->data.name << endl;
        e = e->next;
    }
    cout << endl;
}
```

```
int main(){
    List* list1 = CreateEmptyList();
    Person p[] = {{1,"Sok"}, {2,"Sara"}, {3,"Ken"}, {4,"Long"}};
    int n = sizeof(p)/sizeof(p[0]);

    for(int i=0;i<n;i++){
        function1(list1,p[i]);
    }
    display(list1);
}
```

PRACTICE

- What is the output of the program?

```
#include <iostream>
using namespace std;

struct Person{
    int id;
    string name;
};

struct Element{
    Person data;
    Element* next;
};

struct List{
    int n;
    Element* head;
    Element* tail;
};
```

```
void add(List* ls, int data){
    Element *e = new Element;
    e->data = data;
    e->next = nullptr;
    if(ls->n == 0){ls->head = e;}
    else{ ls->tail->next = e;}
    ls->tail = e;
    ls->n += 1;
}
```

```
void display(List* ls){
    Element* temp = ls -> head;
    cout<<"Number of Element: "<<ls -> n<<endl;
    while(temp != nullptr){
        cout<<temp -> data<<" ";
        temp = temp -> next;
    }
    cout << endl;
}
```

```
int function2(List* ls){
    Element* temp = ls->head;
    int sum = 0;
    while(temp != nullptr){
        sum += temp->data;
        temp = temp->next;
    }
    return sum;
}
```

```
int main(){
    List* list1 = createEmptyList();
    for(int i=0;i<=5;i++){
        if(i%2==1){
            add(list1,i);
        }
    }
    cout<<function2(list1)<<endl;
}
```

PRACTICE

1. Create a singly linked list that can store integer numbers. Then
   a. Create two data structure (Element and List), and create an empty singly linked list
   b. Create function to add the element to the end of the list
   c. Create function to add the element to the beginning of the list
   d. Create function to display all numbers in the list
   e. Create function to add the element before position p of the list
   f. Create function to add the element after the position p of the list

PRACTICE

2. Create a singly linked list that can store id, names of students.
    a. Create structure of student, element, list, and create an empty singly linked list.
    b. Create a function to add the student's info to the beginning of the list
    c. Create a function to add the student's info to end of the list
    d. Display all students in the list
    e. Create a function to search by student id in the list and display the student's name if found.
    f. Create a function to Update the name for the input student id

Q&A